SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 40%

QUESTIONS:5 **Total Marks:100**

Annexure A

EXPERIMENTS

Code of Conduct:

I D K SUMALATHA(192424023) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

D K SUMALATHA

Q1. Desk Calculator Using Syntax-Directed Definition (SDD)

Aim

To implement a desk calculator using **Syntax-Directed Definition (SDD)** and synthesized attributes to evaluate arithmetic expressions containing +, -, *, /, and parentheses while maintaining operator precedence.

Algorithm

1. Start the program.
2. Read an arithmetic expression from the user.
3. Define grammar functions for:
    - E → Expression
    - T → Term
    - F → Factor
4. Evaluate multiplication and division at the T level.
5. Evaluate addition and subtraction at the E level.
6. Evaluate numbers and parenthesized expressions at the F level.
7. Use the returned value of each function as a **synthesized attribute**.
8. Apply the semantic rules while parsing.
9. Display the final calculated value.
10. Stop.

C Program

```
#include <stdio.h>
#include <ctype.h>

char expr[100];
int pos = 0;

/* Function prototypes */
float E();
float T();
float F();
```

```
/* E -> E + T | E - T | T */
float E()
{
   float value = T();

   while (expr[pos] == '+' || expr[pos] == '-')
   {
      char op = expr[pos++];
      float value2 = T();

      if (op == '+')
         value = value + value2;
      else
         value = value - value2;
   }

   return value;
}

/* T -> T * F | T / F | F */
float T()
{
   float value = F();

   while (expr[pos] == '*' || expr[pos] == '/')
   {
      char op = expr[pos++];
      float value2 = F();

      if (op == '*')
```

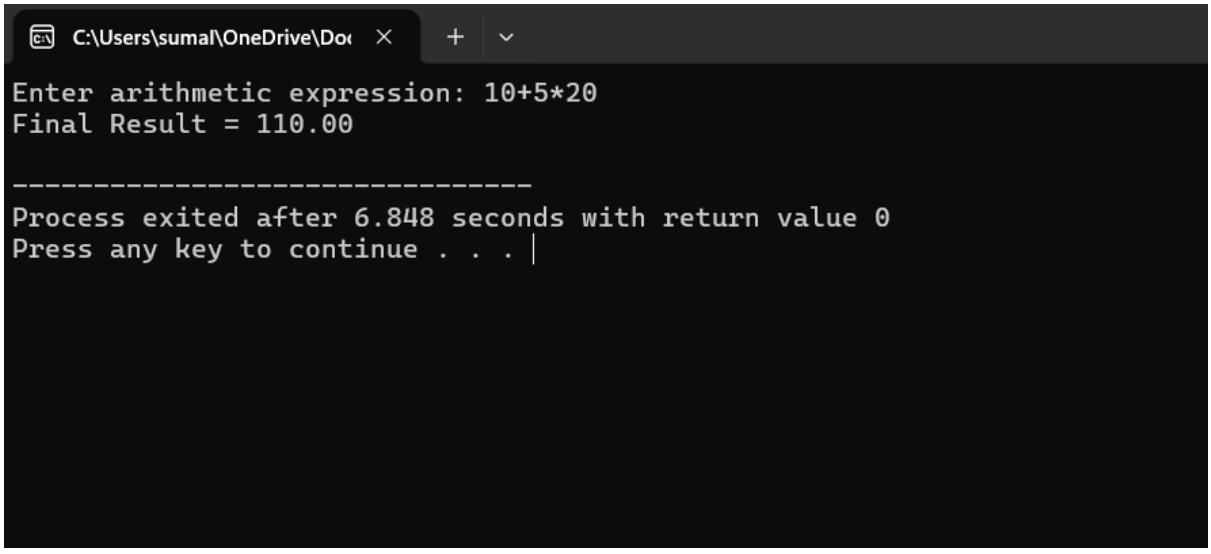
```
            value = value * value2;
        else
            value = value / value2;
    }

    return value;
}

/* F -> (E) | digit */
float F()
{
    float value = 0;

    if (expr[pos] == '(')
    {
        pos++;
        value = E();
        pos++;   // Skip ')'
    }
    else
    {
        while (isdigit(expr[pos]))
        {
            value = value * 10 + (expr[pos] - '0');
            pos++;
        }
    }

    return value;
}
```

```
int main()
{
    printf("Enter arithmetic expression: ");
    scanf("%s", expr);

    float result = E();

    printf("Final Result = %.2f\n", result);

    return 0;
}
```

Sample Input

10+5*20

Sample Output



SDD Concept

The synthesized attribute is the value returned by each grammar function.

For example:

E → E + T { E.val = E1.val + T.val }

E → E - T { E.val = E1.val - T.val }

E → T { E.val = T.val }

$T \rightarrow T * F$ { T.val = T1.val * F.val }

$T \rightarrow T / F$ { T.val = T1.val / F.val }

$T \rightarrow F$ { T.val = F.val }

$F \rightarrow (E)$ { F.val = E.val }

$F \rightarrow digit$ { F.val = digit }

Here, values are synthesized from the child nodes to the parent node.

Result

Thus, the arithmetic expression was successfully evaluated using **Syntax-Directed Definition with synthesized attributes**, and operator precedence was correctly maintained.

Q2. Postfix Expression Evaluation Using S-Attributed Definition

Aim

To implement a C program for evaluating a postfix expression using **stack-based bottom-up evaluation** and demonstrate an **S-attributed definition** with intermediate computations.

Algorithm

1. Start the program.
2. Read the postfix expression.
3. Create an empty stack.
4. Scan the expression from left to right.
5. If the symbol is an operand, push it onto the stack.
6. If the symbol is an operator:
    - Pop the second operand.
    - Pop the first operand.
    - Perform the operation.
    - Push the intermediate result back onto the stack.
7. Continue until the expression ends.
8. The remaining stack value is the final result.
9. Display the result.
10. Stop.

C Program

```
#include <stdio.h>
#include <ctype.h>

int stack[100];
int top = -1;

void push(int value)
{
   stack[++top] = value;
}

int pop()
{
   return stack[top--];
}

int main()
{
   char postfix[100];
   int i;
   int a, b, result;

   printf("Enter postfix expression: ");
   scanf("%s", postfix);

   for (i = 0; postfix[i] != '\0'; i++)
   {
      if (isdigit(postfix[i]))
      {
```

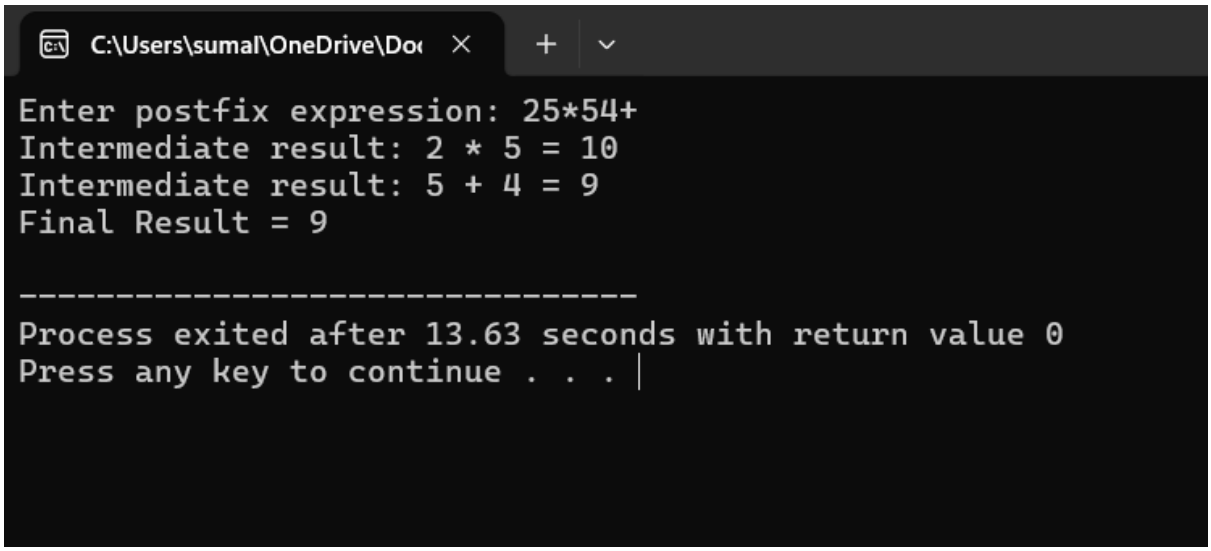
```
    push(postfix[i] - '0');
}
else
{
    b = pop();
    a = pop();

    switch (postfix[i])
    {
        case '+':
            result = a + b;
            break;

        case '-':
            result = a - b;
            break;

        case '*':
            result = a * b;
            break;

        case '/':
            result = a / b;
            break;
    }

    printf("Intermediate result: %d %c %d = %d\n",
          a, postfix[i], b, result);

    push(result);
```

```
        }
    }

    printf("Final Result = %d\n", pop());

    return 0;
}
```

Sample Input

25*54*+

Sample Output



S-Attributed Concept

An S-attributed definition uses only **synthesized attributes**.

For example:

E → E1 + E2

E.val = E1.val + E2.val

The values are calculated from the child nodes and passed upward during bottom-up evaluation.

Result

Thus, the postfix expression was successfully evaluated using a **stack-based S-attributed bottom-up method**, and intermediate results were displayed.

Q3. Expression Evaluation Using L-Attributed Definition

Aim

To implement a C program that evaluates an expression such as a + b * c using **function parameters to simulate inherited attributes**, demonstrating L-attributed top-down evaluation and correct operator precedence.

Algorithm

1. Start the program.
2. Assign values to variables a, b, and c.
3. Define functions for Expression, Term, and Factor.
4. Pass the accumulated value as a function parameter.
5. Use the parameter to simulate an **inherited attribute**.
6. Evaluate multiplication before addition.
7. Pass intermediate values from parent functions to child functions.
8. Calculate the final expression value.
9. Display the result.
10. Stop.

C Program

```
#include <stdio.h>


/*
  L-attributed evaluation is simulated by
  passing inherited values as function parameters.
*/


int a, b, c;


/* Factor */
int F(char variable)
{
   if (variable == 'a')
      return a;
   else if (variable == 'b')
      return b;
```

```
    else
        return c;
}

/* Term with inherited value */
int T_prime(int inherited)
{
    return inherited;
}

/* Expression */
int E(int inherited)
{
    int termResult;

    termResult = T_prime(inherited);

    return termResult;
}

int main()
{
    int result;
    int bxc;

    printf("Enter value of a: ");
    scanf("%d", &a);

    printf("Enter value of b: ");
    scanf("%d", &b);
```

```
    printf("Enter value of c: ");
    scanf("%d", &c);

    /* b * c is evaluated first */
    bxc = F('b') * F('c');

    /* inherited value passed to E */
    result = E(F('a') + bxc);

    printf("\nExpression: a + b * c\n");
    printf("Result = %d\n", result);

    return 0;
}
```

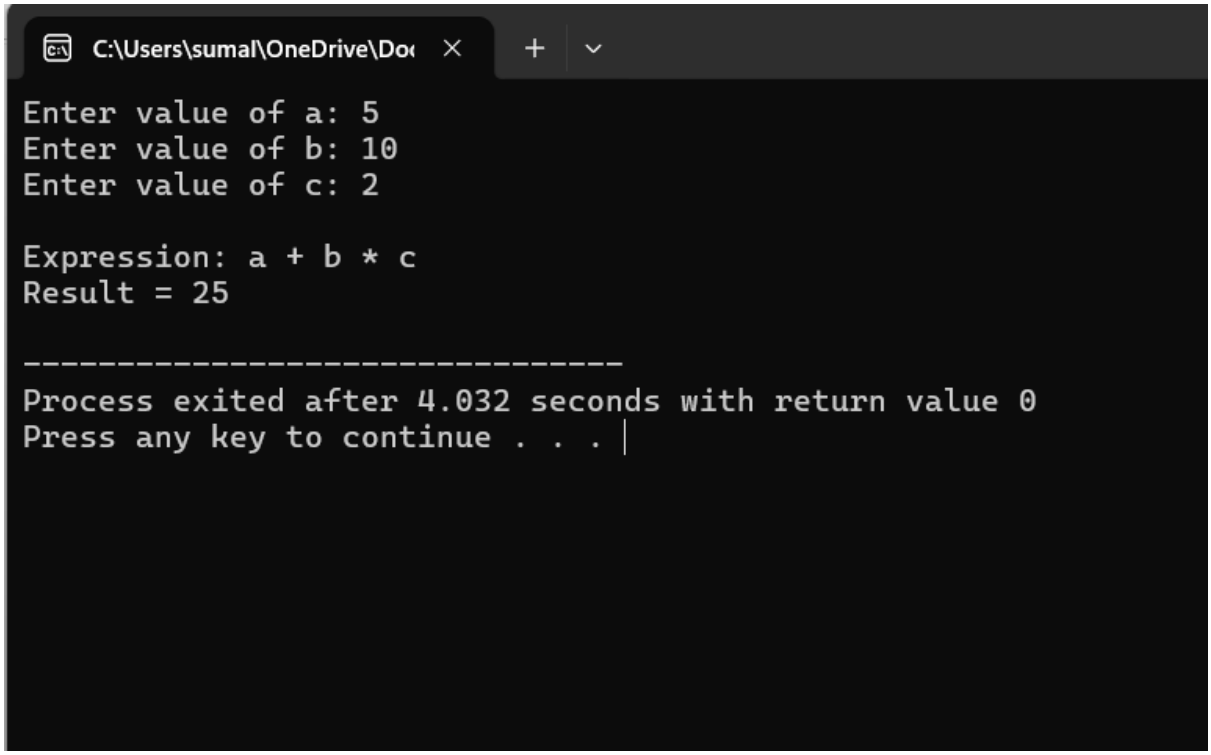
Sample Input

Enter value of a: 10

Enter value of b: 5

Enter value of c: 2

Sample Output



Result

Thus, the expression was successfully evaluated using **L-attributed evaluation**, where function parameters simulate inherited attributes and correct operator precedence was maintained.

Q4. Type Expression Equivalence

Aim

To implement a C program that accepts two type expressions and checks whether they are **equivalent or not equivalent**, including pointer types such as int* and float*.

Algorithm

1. Start the program.
2. Read the first type expression.
3. Read the second type expression.
4. Compare the two type expressions.
5. If both are exactly the same, they are equivalent.
6. For pointer types:
    - Extract the base type.
    - Check whether both are pointers.
    - Compare their base types.
7. If the base types are the same, the pointer types are equivalent.

8. Otherwise, report that they are not equivalent.
9. Display the result.
10. Stop.

C Program

```
#include <stdio.h>
#include <string.h>

int isEquivalent(char type1[], char type2[])
{
   int len1 = strlen(type1);
   int len2 = strlen(type2);

   /* Direct comparison */
   if (strcmp(type1, type2) == 0)
      return 1;

   /* Pointer type comparison */
   if (len1 > 1 && len2 > 1 &&
      type1[len1 - 1] == '*' &&
      type2[len2 - 1] == '*')
   {
      type1[len1 - 1] = '\0';
      type2[len2 - 1] = '\0';

      if (strcmp(type1, type2) == 0)
         return 1;
   }

   return 0;
}
```

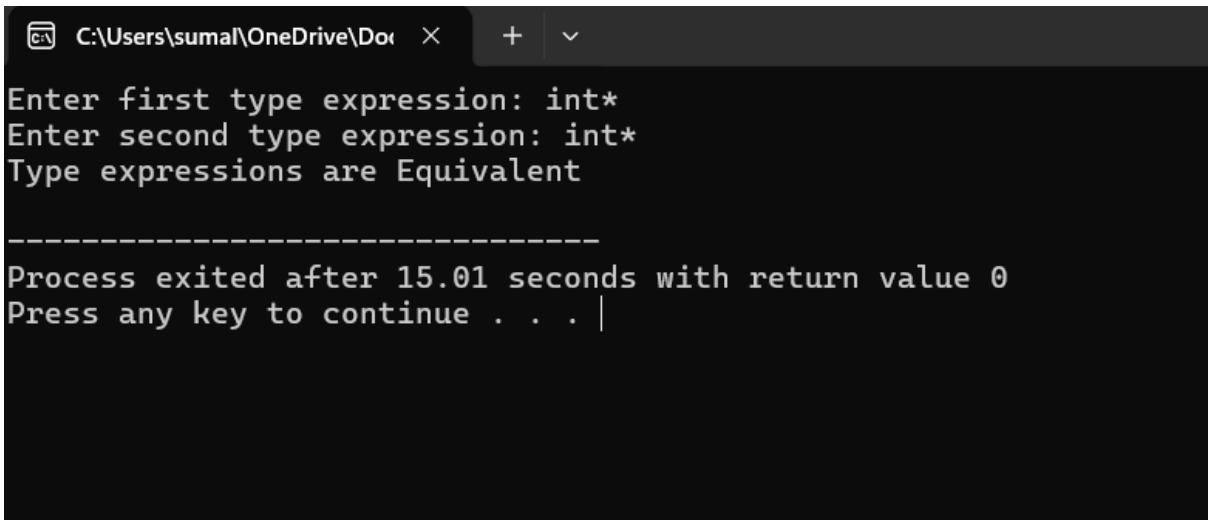
```
int main()
{
    char type1[20], type2[20];

    printf("Enter first type expression: ");
    scanf("%s", type1);

    printf("Enter second type expression: ");
    scanf("%s", type2);

    if (isEquivalent(type1, type2))
        printf("Type expressions are Equivalent\n");
    else
        printf("Type expressions are Not Equivalent\n");

    return 0;
}
```

Sample Input

Enter first type expression: int*

Enter second type expression: int*

Sample Output



Type expressions are Equivalent

Result

Thus, the program successfully compared type expressions and identified whether basic types and pointer types were **equivalent or not equivalent**.

Q5. Semantic Analyzer for Type Checking

Aim

To implement a C program that performs **semantic type checking** on arithmetic expressions and detects invalid operations such as int + char*.

Algorithm

1. Start the program.
2. Read the first operand and its type.
3. Read the second operand and its type.
4. Read the arithmetic operator.
5. Check whether the operands have valid types for the operation.
6. Allow arithmetic operations between compatible numeric types such as:
    - int + int
    - int * int
    - float + float
    - int + float
7. Reject operations involving pointer types in this simple analyzer.
8. If the operation is valid, display **Valid expression**.
9. Otherwise, display **Type error**.
10. Stop.

C Program

```
#include <stdio.h>
#include <string.h>

int isNumeric(char type[])
{
   if (strcmp(type, "int") == 0 ||
       strcmp(type, "float") == 0 ||
```

```
        strcmp(type, "double") == 0)
        return 1;

    return 0;
}

int main()
{
    char operand1[20], operand2[20];
    char type1[20], type2[20];
    char operator;

    printf("Enter first operand: ");
    scanf("%s", operand1);

    printf("Enter type of first operand: ");
    scanf("%s", type1);

    printf("Enter operator (+, -, *, /): ");
    scanf(" %c", &operator);

    printf("Enter second operand: ");
    scanf("%s", operand2);

    printf("Enter type of second operand: ");
    scanf("%s", type2);

    if (isNumeric(type1) && isNumeric(type2))
    {
        printf("\nValid expression\n");
```

```
        printf("Semantic type checking successful.\n");
    }
    else
    {
        printf("\nType error\n");
        printf("Incompatible operand types.\n");
    }

    return 0;
}
```

Sample Input 1 – Valid Expression

Enter first operand: a

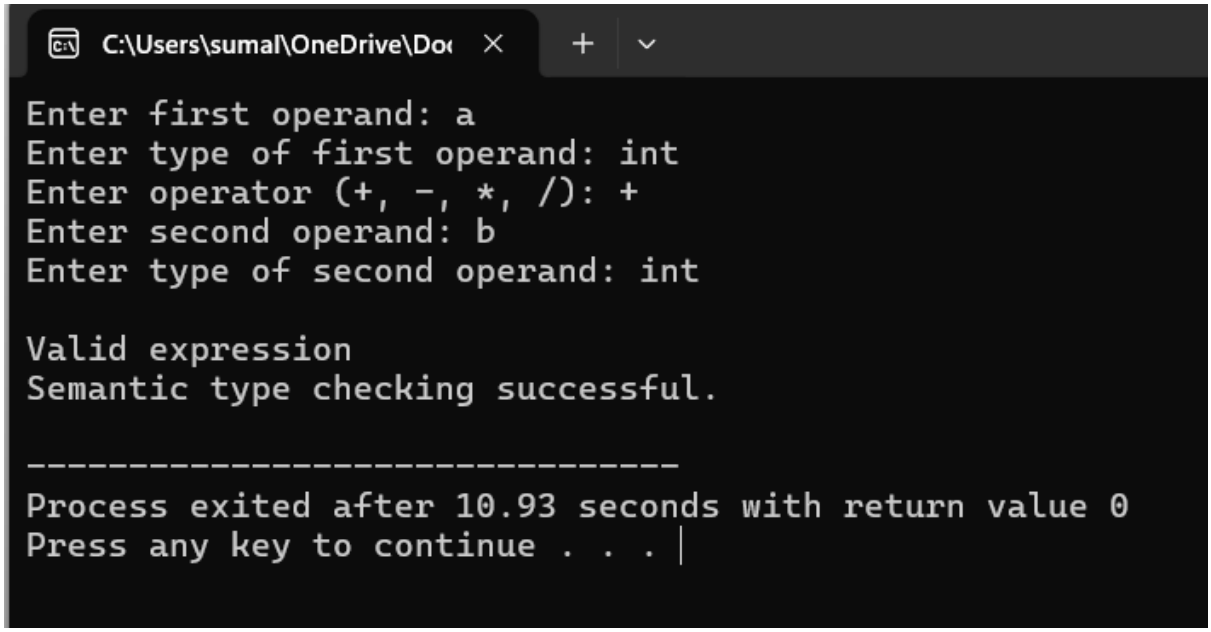
Enter type of first operand: int

Enter operator (+, -, *, /): +

Enter second operand: b

Enter type of second operand: int

Sample Output



Valid expression

Semantic type checking successful.

Semantic Analysis Concept

Semantic analysis checks whether an expression is meaningful according to the language's type rules.

For example:

int + int

is valid.

But:

int + char*

is invalid because an integer cannot be directly added to a character pointer in this simplified type system.

Result

Thus, the semantic analyzer successfully checked operand types and detected **valid expressions and semantic type errors** during compiler analysis.